malloc() function

malloc() in C+ is a function that allocates memory for variables or objects on the heap.

It accepts size (memory space required by the variable) in bytes as an argument and returns a pointer to the allocated memory location.

The syntax for malloc() function is:

malloc(<size_in_bytes>)

Properties of malloc() function

- malloc() returns an untyped pointer (i.e. void*). The return type needs to be explicitly typecasted into the type of the object.
- malloc() returns a null pointer if the memory allocation fails for some reason.
- Creating an object using malloc() doesn't invoke the constructor of the class.
- Memory allocated by malloc() can be deallocated using the free() method.
- The use of the free() method for destroying an object doesn't invoke the destructor of the class.

Example :

Creating a Dynamic array of 5 integers using malloc in c++.

#include<iostream>

#include<cstdlib>

Vector India

```
int main()
{
int size;
int *array=NULL;
cout<<"Enter the size:"<<endl;
cin>>size;
array=(int *)malloc(size*sizeof(int));
cout<<"Enter the elements:"<<endl;
/*** inputting elements *****/
for(int i=0;i<size;i++)
cin>>array[i];
/**** displaying elements ****/
cout<<"Elements of an array:"<<endl;
for(int i=0;i<size;i++)
cout<<array[i]<<" ";
cout<<endl;

/**** Deallocating  memory *****/
free(array);
```

Vector India

```
array=NULL;
}
```

Output :

```
Enter the size:
5
Enter the elements:
2 4 6 8 10
Elements of an array:
2 4 6 8 10
```

new operator

The new operator also allocates memory for variables or objects on the heap.

The new operator automatically calculates the size of the memory block for the given data type.

For multiple blocks of memory allocation (like in the case of an array), the number of memory blocks needed for allocation should be provided at object declaration.

Vector India

The simple syntax for new operator is:

new datatype; // single data

Example :

int *ptr=new int;

char *ch=new char;

Assume struct Student as a user defined datatype

struct Student *S=new struct Student;

new datatype[size]; // creates an array of size elements

Example :

int *ptr=new int[5]; // creates the array of 5 integer

struct Student *s=new struct Student[5];

Properties of new Operator

- new operator returns strictly typed pointer. Hence it doesn't require typecasting.
- new doesn't return a null pointer if the memory allocation fails, rather it returns zero or throws an exception.
- When new is used to allocate memory for a C++ class object, the object's constructor is called after the memory is allocated

Vector India

- Memory allocated by new can be deallocated using the delete operator.
- The use of the delete operator on class objects also invokes the destructor of the class.

Example :
Creating a Dynamic array of 5 integers using new in c++.

```
#include<iostream>
int main()
{
int size;

int *array=NULL;
cout<<"Enter the size:"<<endl;
cin>>size;
array=new int[size];
cout<<"Enter the elements:"<<endl;
/*** inputting elements *****/
for(int i=0;i<size;i++)
cin>>array[i];
/**** displaying elements ****/
cout<<"Elements of an array:"<<endl;
for(int i=0;i<size;i++)
cout<<array[i]<<" ";
cout<<endl;

/**** Deallocating  memory *****/
delete[]array;
array=NULL;
}
```

Vector India

Output :
Enter the size:
5
Enter the elements:
2 4 6 8 10
Elements of an array:
2 4 6 8 10

Note: delete array will deallocate memory block for only the first element of the array. To deallocate the heap memory used by the whole array use delete[] array instead.

malloc Vs new

malloc	**new**
malloc is a predefined function	new is an operator
Syntax : void * malloc(size_t);	new datatype
Returns void *	Returns strictly typed pointer
Allocated memory can be deallocated by free()	Allocated memory can be deallocated by delete
Returns NULL upon failure	Throws an exception upon failure
malloc cannot invoke Constructor	new can invoke constructor
malloc cannot be overloaded	new can be overloaded
malloc cannot initialize at the time of memory allocation	new can initialize at the time of memory allocation.
Memory allocated by malloc can be reallocated using realloc()	Memory allocated by new cannot be reallocated using realloc
free cannot invoke the destructor	delete cannot invoke the destructor

Vector India